

C05 - Assessment Tasks 2

Error Analysis Task

SET1

1. **A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.**

ANSWER:

Hadoop Job Repeatedly Fails Because Input Splits Are Uneven Problem

In Hadoop, a large input file is divided into smaller input splits. Each split is generally processed by one Mapper.

If the input splits are not distributed evenly, some Mappers may receive very large amounts of data while others receive very little data. The Mappers with smaller workloads finish quickly, while the Mappers handling large splits continue running. This creates a situation called straggler tasks, where a few tasks take much longer than the others. Reducers may remain idle because they are waiting for mapper output.

Likely Design Error

The likely design error is poor input partitioning or data distribution.

For example:

- Mapper 1 → 1 GB
- Mapper 2 → 1 GB
- Mapper 3 → 10 GB
- Mapper 4 → 1 GB

Mapper 3 takes much longer than the others. Reducers that depend on its output have to wait.

Corrective Actions

1. Choose an appropriate HDFS block size
HDFS should divide large files into reasonably sized blocks.
2. Configure input split size properly
Avoid extremely large or extremely small splits.
3. Balance input files
If possible, avoid having a few very large files while many files are very small.
4. Handle data skew
If particular keys contain much more data than others, use a suitable partitioning strategy.
5. Use a proper Partitioner
Hadoop's Partitioner determines which reducer receives a particular key. A good partitioner distributes keys evenly.
6. Monitor Mapper and Reducer performance
Identify tasks that consistently take much longer than others.

Example

Suppose there are 4

Mappers: Input Data

|

+ Split 1 $\rightarrow$ Mapper 1 $\rightarrow$ 2 GB

- + Split 2 → Mapper 2 → 2 GB
- + Split 3 → Mapper 3 → 2 GB
- + Split 4 → Mapper 4 → 2 GB

This is much better than giving one Mapper 10 GB and the others 1 GB.

Conclusion

The input data should be evenly distributed among Mappers, and the output should be properly partitioned among Reducers. This improves parallelism and reduces job execution time.

2. **An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.**

ANSWER:

HDFS Data Becomes Unavailable After a Single Node Failure Problem:

HDFS is designed to store multiple copies of data. These copies are called replicas.

If a DataNode fails and the data stored on that node has no other replica, users cannot access that data.

For example:

File A

```
|
+ DataNode 1
```

If DataNode 1 fails:

DataNode 1 

```
|
└── File A unavailable
```

Likely Design Error

The most likely problem is an insufficient replication factor.

If the replication factor is set to 1, only one copy of each block exists.

A better configuration is:

File Block

```
|
+ DataNode 1
+ DataNode 2
+ DataNode 3
```

Now, if DataNode 1 fails, the other two copies are still available.

Another possible design error is having only one NameNode without High Availability (HA).

Corrective Actions

1. Increase the replication factor For example:

Replication Factor = 3

This creates three copies of each HDFS block.

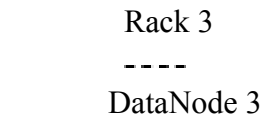
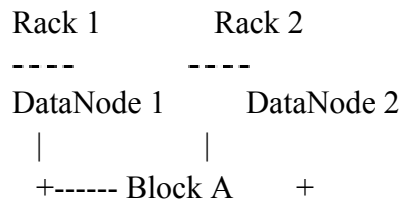
2. Use different DataNodes

Copies should not all be stored on the same physical machine.

3. Use rack awareness

Hadoop can distribute replicas across different racks.

For example:

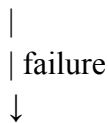


This protects data even if an entire rack fails.

4. Configure NameNode High

Availability Use:

Active NameNode



Standby NameNode

If the Active NameNode fails, the Standby NameNode can take over.

5. Monitor under-replicated blocks

HDFS should automatically create new replicas when a DataNode fails.

Conclusion

The main solution is proper replication + NameNode High Availability. This ensures that a single hardware failure does not make the data unavailable.

3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

ANSWER:

ETL and Apache NiFi Creates Duplicate Records

Problem

Apache NiFi is commonly used to collect, transform and move data between systems.

For example:

Source



Apache NiFi



ETL Processing



Analytics Database

Sometimes the same record may enter the analytics database multiple times.

Example

Suppose the original data is:

Customer ID | Name

-----|-----

101 | Ravi

102 | Priya

Due to duplicate processing, the database becomes:

Customer ID	Name
101	Ravi
101	Ravi
102	Priya
102	Priya

Likely Causes

1. Same file processed multiple times

NiFi may receive the same file again after a retry or system failure.

2. Retry after failure

Suppose NiFi successfully sends a record to the database but does not receive confirmation.

It may assume that the operation failed and send the record again.

Send Record



Database receives record ✓



Response lost ✗



NiFi retries



Same record inserted again

3. No unique identifier

If records do not have a unique ID, the destination system may not know that two records are actually the same.

4. No duplicate detection

The NiFi flow may not contain mechanisms to identify already processed records.

5. Database allows duplicates

The destination table may not have a PRIMARY KEY or UNIQUE constraint.

Corrective Actions

1. Use unique IDs

Every record should have a unique identifier.

Example:

Transaction ID = TX1001

2. Use duplicate detection

NiFi can use suitable duplicate-detection mechanisms, such as the DetectDuplicate processor, with a distributed state/cache where appropriate.

3. Use database

constraints For example:

Transaction_ID → UNIQUE

This prevents the same transaction from being inserted twice.

4. Use UPSERT/MERGE

Instead of always inserting:

INSERT

use logic such as:

If record exists → UPDATE

If record does not exist → INSERT

5. Track processed files

Maintain information about which files or events have already been processed.

Conclusion

Duplicate records are usually caused by reprocessing, retries, lack of unique IDs, or missing duplicate detection. Using unique keys, idempotent processing, duplicate detection and database constraints solves the problem.

4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

ANSWER

:

YARN Cluster Suffers From Resource Starvation

Problem

YARN manages computing resources in a Hadoop cluster.

It manages resources such as:

- CPU
- Memory
- Containers

Suppose there are multiple users:

User A → submits 10 jobs

User B → submits 5 jobs

User C → submits 3 jobs

If User A consumes almost all available resources, Users B and C may have to wait.

This is called resource starvation.

Likely Design Error

The cluster may not have a proper resource scheduling policy.

For example:

Total Cluster Memory = 100 GB

User A → 90 GB

User B → 5 GB

User C → 5 GB

User A is consuming most of the cluster resources.

Corrective Actions

1. Use Capacity Scheduler

The Capacity Scheduler divides cluster resources into queues.

For example:

Hadoop Cluster

```
|
+   Research Queue → 40%
|
+---- Student Queue → 30%
|
+   Production Queue → 30%
```

Each queue receives a guaranteed portion of resources.

2. Set queue limits

Prevent one queue from consuming all cluster resources.

3. Set user limits

Set appropriate resource limits for individual users.

4. Use Fair Scheduling

Fair scheduling attempts to distribute resources fairly among running applications/users.

5. Set job priorities

Important jobs can be given higher priority when appropriate.

6. Monitor ResourceManager

The YARN ResourceManager should be monitored for:

- CPU utilization
- Memory utilization
- Running applications
- Queue usage
- Waiting applications

Example

Without proper scheduling:

User A →  90%

User B →  5%

User C →  5%

With proper queue management:

User A →  40%

User B →  30%

User C →  30%

Conclusion

The solution is to use a proper YARN scheduler with queues, resource limits and fair allocation. Capacity Scheduler is particularly suitable when organizations need guaranteed resources for different users or departments.

5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

ANSWER:

Backup Restores Outdated Data After Recovery

Problem

A distributed storage system may contain several copies of data. However, replication and backup are not the same thing.

Replication keeps copies of current data, while backup maintains recoverable versions of data over time.

Suppose:

Monday → Backup A

Tuesday → Backup B

Wednesday → Backup C

Thursday → Data corruption

If the system restores Backup A instead of the latest valid backup, the organization loses the changes made on Tuesday and Wednesday.

Likely Design Flaw

The backup system may be:

- Using old backups

- Not maintaining backup versions
- Not recording timestamps
- Overwriting previous backups
- Not verifying backup integrity
- Treating replication as a complete backup

solution Important Point

Replication alone does not protect against every type of data loss.

For example:

Primary Data



Replic

a

If the primary data is accidentally deleted:

Primary → Deleted ❌



Replica → Delete replicated ❌

Both copies may now be missing the data.

Therefore, independent backups are important.

Corrective Actions

1. Maintain multiple backup

versions For example:

Backup 1 → Monday

Backup 2 → Tuesday

Backup 3 → Wednesday

Backup 4 → Thursday

The administrator can select the latest valid backup.

2. Use timestamps

Every backup should have:

Backup Date

Backup Time

Backup Version

3. Take regular snapshots

Snapshots provide point-in-time recovery.

4. Verify backup integrity

Regularly check whether backup data can actually be restored.

5. Keep backups independent

Do not store all backup copies on the same infrastructure as the primary data.

6. Follow the 3-2-1 backup

principle A common strategy is:

3 copies of data



2 different storage media



1 copy stored off-site

7. Regularly test recovery

A backup is useful only if it can actually be restored.

Conclusion

The backup system should maintain multiple, timestamped, verified versions of data and should be separate from normal replication. This ensures that the latest valid data can be recovered after failure or corruption.